

Forest 项目架构整理

=====

整理日期：2026-04-07

工作区：/Users/lgd/project/forest

一、项目的设计目的

这个项目的核心目标，不是简单把前端和后端代码放进同一个仓库，而是建立一套可以长期演进的“业务模块化”体系。

它的设计目的可以概括为：

1. 把多样的业务能力沉淀到 `business` 目录中。
2. 把多个 `business` 模块组合成不同 app，放在 `apps` 目录中。
3. 把基础设施沉淀在 `base-frontend` 和 `base-backend` 中。
4. 通过结构设计追求业务高内聚、模块低耦合。
5. 在前端层面实现“业务逻辑与 UI 分离”。

二、整体技术形态

从纯技术视角看，这个项目是：

- 一个按业务域纵切的 `modular monolith`
- 外面套了一层 `monorepo` 工程体系
- 而不是微服务架构
- 也不是传统的前后端分仓 + `shared` 大杂烩模式

技术栈大致如下：

- 后端：Maven 多模块、Spring Boot、Spring Data JPA、PostgreSQL、Redis、JWT
- 前端：pnpm workspace、Turborepo、Vue 3、Vite、TypeScript

三、顶层目录的技术职责

1. `business/`
这是业务模块层。
每个业务模块都应当是“完整业务能力单元”，前后端成对存在。
2. `apps/`
这是应用组合层。
它的职责不是沉淀通用业务，而是组合多个 `business` 模块形成具体产品。
3. `base-backend/`
这是后端基础设施层。
提供父 POM、starter、统一异常与返回结构等公共能力。
4. `base-frontend/`
这是前端基础设施层。
提供 workspace、turbo、统一依赖、基础 UI kit 等公共能力。
5. `docs/`
这是知识与设计文档层。
存放架构说明、产品设计、重构计划与团队共享知识。
6. `docker/`
这是本地运行环境层。
用于提供数据库等基础依赖的启动方式。

四、目录分层背后的技术原则

这个仓库的关键不是“代码放在哪里”，而是“每一层允许做什么，不允许做什么”。

1. business 允许做什么

- 定义自己的后端实体、Repository、Service、Controller
- 定义自己的前端 types、api、composables、各端 UI
- 形成一个可被多个 app 重复组合的完整业务模块

business 不应该做什么：

- 依赖某个具体 app
- 依赖太多别的 business，形成相互缠绕
- 把通用工程基础设施塞进自己模块里

2. apps 允许做什么

- 组合多个 business 模块
- 做跨模块业务编排
- 放应用启动入口、应用级配置、应用级页面壳

apps 不应该做什么：

- 持续长出大块可复用业务逻辑
- 反向成为 business 的基础依赖
- 取代 business 成为真实业务沉淀层

3. base-* 允许做什么

- 放所有模块都依赖的最小公共能力
- 放工程层约定、starter、基础 UI、构建配置

base-* 不应该做什么：

- 放 user/account/leads/points 这类业务逻辑
- 长成一个新的“共享业务层”

五、后端架构理解

后端是一个典型的三层结构：

1. base-backend：基础设施层
2. business/*/backend：业务域层
3. apps/*/backend：应用组合层

5.1 base-backend 的作用

~~~~~

base-backend/pom.xml

- 这是后端模块的父 POM
- 统一管理 Spring Boot、Java 版本、插件与内部模块依赖

base-backend/starter-common

- 这是内部 starter
- 它承载的是跨业务通用基础设施

当前已经看到的公共内容包括：

- Result<T>：统一接口返回结构
- BusinessException：统一业务异常
- ForestAutoConfiguration：自动配置入口

这层的技术意义是：  
把“工程公共约定”统一起来，让 business 模块尽量只关心业务本身。

## 5.2 business/user/backend 的作用

这是用户域模块。

当前结构：

- entity/User.java
- repository/UserRepository.java
- service/UserService.java
- controller/UserController.java

技术上它代表的是“用户业务域”。

它的职责应该是：

- 保存用户资料
- 提供用户查询与创建接口
- 作为一个可被 app 使用的标准业务模块

从设计目标看，这个模块未来更适合承载“用户档案/个人资料”，  
而不应该长期承担认证和积分等其他业务域职责。

## 5.3 business/account/backend 的作用

这是认证域模块，也是当前完成度较高的一个 business 模块。

它的关键点在于：  
项目把“账号/登录凭证”和“用户资料”拆成了两个并列概念。

当前结构包括：

- entity/Account.java
- entity/SmsCode.java
- repository/AccountRepository.java
- repository/SmsCodeRepository.java
- service/AccountService.java
- service/AuthService.java
- service/JwtTokenProvider.java
- service/PasswordEncoder.java
- service/WechatService.java
- service/WechatSessionService.java
- config/AuthConfig.java
- config/AuthInterceptor.java
- context/AccountContext.java
- controller/AuthController.java

这说明 account 模块的定位是：

- 管理登录凭证
- 提供登录认证能力
- 提供 token 生成与校验能力
- 提供请求期身份上下文

也就是说，这个模块在技术上是“认证基础业务域”，  
而不是 user 模块的一个附属子功能。

## 5.4 apps/admin/backend 的作用

这是管理后台应用的后端入口。

它按设计应该负责：

- 启动 Spring Boot 应用
- 装配多个 business 后端模块
- 放 admin 应用级配置
- 在需要时承担跨 business 的应用级编排

它的技术角色是 composition root，而不是通用业务沉淀层。

## 5.5 apps/trade-leads/backend 的作用

~~~~~

这是外贸线索产品的应用后端。

从当前代码看，它还承载了不少 leads / unlock / point 相关实现。
但从文档目标看，它最终应该进一步瘦身：

- leads 应拆成独立 business 模块
- points 应拆成独立 business 模块
- app 只保留跨模块编排逻辑

因此它现在的技术状态更像“过渡态应用层”。

六、后端包路径的技术意义

在 business 模块内部，包路径不是随便分的，而是有明确技术角色。

1. entity

- 表达持久化实体
- 对应数据库表和领域状态
- 不负责跨模块流程编排

2. repository

- 负责数据访问
- 封装数据库查询和持久化操作
- 不应承担复杂业务逻辑

3. service

- 负责当前模块内部业务规则
- 是该业务域的核心逻辑实现层

4. controller

- 是模块对外 API 边界
- 负责接收请求、调用 service、返回结果

5. config

- 用于模块级 Spring 配置
- 例如拦截器、Bean、过滤规则

6. context

- 用于保存请求级上下文
- 例如当前登录账号信息

7. dto

- 用于请求与响应模型
- 用来隔离实体结构与接口表达

七、前端架构理解

前端的设计重点不是页面，而是“业务能力包”。

这个项目不是先定义一个 app，再在 app 里慢慢长页面逻辑，而是先把业务能力抽成模块，再由 app 去消费这些能力。

7.1 base-frontend 的作用

base-frontend 是前端基础设施根目录。

它的职责包括：

- 统一依赖版本
- 定义 turbo 任务
- 配置 pnpm workspace
- 提供基础 UI kit

其中关键文件包括：

- package.json
- pnpm-workspace.yaml
- turbo.json
- packages/ui-kit/*

技术意义是：

把前端 monorepo 的工程控制面统一起来。

7.2 base-frontend/packages/ui-kit 的作用

这是基础 UI 组件层。

比如当前已有：

- FButton.vue

这类组件不带具体业务语义，
它们是所有业务模块都可以使用的基础组件。

这层不应该出现 user、account、leads 这类业务概念。

7.3 business/user/frontend 的作用

这是用户域前端能力包。

它通过 package exports 对外暴露业务能力，而不是只暴露一个单一入口。

当前结构包括：

- src/types.ts
- src/api.ts
- src/composables.ts
- src/ui/admin/*
- src/ui/mobile/*

技术角色是：

- types.ts: 定义用户模块前端契约
- api.ts: 封装用户模块后端调用
- composables.ts: 封装用户业务逻辑
- ui/admin、ui/mobile: 封装不同端的界面表达

7.4 business/account/frontend 的作用

这是认证域前端能力包。

它体现了项目在前端层面的核心抽象：

业务逻辑和 UI 分离。

当前结构包括：

- src/types.ts
- src/api.ts
- src/composables.ts
- src/ui/admin/*
- src/ui/mobile/*
- src/ui/mp-weixin/*

这说明：

- 认证逻辑尽量收敛在 types/api/composables 中
- 不同端的 UI 差异由 ui/<platform> 承担

因此同一个业务模块可以同时服务：

- 管理后台
- 移动端 H5
- 微信小程序

7.5 apps/admin/frontend 的作用

这是 admin 应用的前端入口。

它的角色应该是：

- 消费 @forest/user、@forest/account 这类业务能力包
- 负责 app 级布局、页面装配、路由与产品壳

这层不应该反过来沉淀大量可复用业务逻辑，否则会破坏业务模块化的边界。

八、前端细节抽象：业务逻辑与 UI 分离

这是整个前端设计里最关键的一条。

业务模块前端按如下方式组织：

- types.ts
- api.ts
- composables.ts
- ui/admin
- ui/mobile
- ui/mp-weixin

这套结构的含义是：

1. `types.ts`
定义业务契约
2. `api.ts`
负责和后端交互
3. `composables.ts`
负责状态和业务逻辑复用
4. `ui/*`
只负责不同端的视觉表达与交互差异

这样做的技术收益是：

- 逻辑可以跨端复用
- UI 可以按端独立演化
- app 不需要重复实现同一套业务逻辑

九、模块组合关系

这个项目真正的复用，不是建立在一个巨大的 `shared` 目录上，而是建立在三层复用结构上。

1. 工程复用

位置：

- `base-backend`
- `base-frontend`

内容：

- `starter`
- 统一依赖
- 构建任务
- 基础 UI
- 统一返回对象
- 公共异常

2. 业务复用

位置：

- `business/*`

内容：

- 完整业务后端模块
- 完整业务前端能力包

3. 产品组合

位置：

- `apps/*`

内容：

- 不同产品场景
- 多 `business` 模块编排
- 应用级入口与配置

十、为什么 `apps` 负责业务编排

当一个业务流程同时涉及多个 `business` 模块时，它本质上已经不是单一业务域内部逻辑，而是应用级流程。

例如：

- 检查账号身份
- 扣减积分
- 记录线索解锁

这类流程同时涉及 `account`、`points`、`leads`。

从技术边界看，它更适合由 `apps` 层负责编排，而不是强行塞到某一个 `business` 中。

这样做有几个明显好处：

1. `business` 模块职责保持纯净
2. 跨模块耦合不会沉到业务基础层
3. `app` 可以按产品场景自由组合多个业务模块
4. 后续重构时，模块边界更清晰

十一、当前实现状态

从当前仓库来看，这套设计理念已经很清晰，但代码还处在“持续落地与收敛”的阶段。

已经比较符合设计目标的部分：

- 已经形成 `base-backend + business backend + apps backend` 的总体结构
- 已经形成 `base-frontend + business frontend + apps frontend` 的总体结构
- `account` 模块已经体现出独立认证域思路
- 前端已经落实 `types/api/composables/ui` 的分层方式

仍在演进中的部分：

- `apps/trade-leads` 仍然偏重，尚未完全瘦身成编排层
- `user` 模块和 `points/leads` 等未来业务域之间的边界仍在调整
- 部分应用层装配逻辑还没有完全按目标架构收敛

也就是说：

这个项目不是“架构没想清楚”，而是“架构目标已经明确，代码还在逐步靠拢”。

十二、最终技术结论

用最简洁的话概括这套架构：

1. `business` 是完整业务模块，要求前后端融合。
2. `apps` 是应用编排层，负责组合多个 `business` 形成具体产品。
3. `base-frontend` 和 `base-backend` 只放最小基础设施。
4. 前端通过 `types/api/composables/ui` 实现业务逻辑和 UI 分离。
5. 整个项目追求的是业务高内聚、模块低耦合、应用可组合、工程可演进。

如果后续开发始终守住这些边界：

- 不让 `apps` 变成新的业务沉淀层
- 不让 `base-*` 长出业务逻辑
- 不让 `business` 之间相互缠绕

那么这个项目就会逐步收敛成一套非常清晰的模块化产品体系。